Gold1

Installation & Handover Dossier

v2.3.3 Minimal Portable Buyer Handover | 12 September 2026

Buyer objective: extract the package, run three commands, obtain exact validated benchmark results with provenance. Fast-prime is optional and cannot block Gold1.

Core setup	./setup_all.sh
Correctness gate	./validate_all.sh
Buyer-grade matrix	GOLD1_AC_QUALIFY_REPEATS=5 RANK_RUNS=5 ./run_interval_matrix.sh
Optional fourth comparator	GOLD1_WITH_FASTPRIME=1 ./setup_all.sh

1. Supported platforms

- Windows reference path: Windows 11/10 using MSYS2 UCRT64. The default path does not require Visual Studio or Rust.
- Linux path: native GCC/Clang/CMake/Ninja build. The package includes prerequisite installers for common distributions.
- The Gold1 mathematical implementation is unchanged from the validated v2.2.3/v2.3.x source snapshot; v2.3.3 changes setup and benchmark orchestration only.

2. Core dependencies

Component	Pinned / requirement	Role
primecount	8.7	Reference endpoint counter and Gold1 integration base
primesieve	12.15	Direct interval comparator and supporting library
Python	3.10+	Benchmark orchestration and provenance
C++ toolchain	C++17 + OpenMP/threads	Gold1 and primecount build
CMake + Ninja	Current supported versions	Reproducible builds
Git/curl/tar	Current supported versions	Pinned source acquisition

Windows core installation is handled inside MSYS2 UCRT64. Linux setup uses the host package manager where supported.

3. Fast-prime is optional

v2.3.3 deliberately removes fast-prime from the blocking dependency chain. Default validation and ranking continue with Gold1, primecount and primesieve when fast-prime is absent.

To include fast-prime V8 as a fourth comparator, run:

```
GOLD1_WITH_FASTPRIME=1 ./setup_all.sh
```

Windows optional path: Visual Studio C++ Build Tools + Rust/MSVC. Linux optional path: rustup/nightly native target. The exact fast-prime commit remains pinned and its compiler/toolchain is recorded when built.

4. Validation and benchmark behavior

- No method is ranked unless its exact count validates.
- Gold1 balanced 6+6 AC is repeatability-qualified. If it is not repeatably exact, Gold1 falls back to sequential 12+12 AC and records the reason.
- Unsupported common-cutoff intervals are recorded as unsupported rather than guessed.
- Optional comparator absence is recorded as optional-skipped and is never treated as a loss or failure.
- Run order rotates between methods to reduce fixed-order bias.

5. Provenance captured per result

- CPU model; physical and logical core counts; installed RAM; OS/version/build.
- UTC benchmark start and finish timestamps.
- Gold1 source-tree SHA-256 and benchmark-harness SHA-256.
- SHA-256 of Gold1, primecount, primesieve, and fast-prime binaries actually used.
- Pinned dependency versions; fast-prime commit and exact Rust toolchain/compiler when present.

6. Acceptance checklist

☐ setup_all.sh exits successfully

☐ validate_all.sh prints ALL_VALIDATION_PASS

☐ matrix creates aggregate CSV and JSON

☐ result metadata identifies the host machine and source/binary hashes

☐ any unsupported/fallback case is explicit in the output

7. Ownership boundary

Commercial transfer should describe the asset as Gold1-authored modifications, integration code, validation/provenance tooling, documentation and know-how. primecount, primesieve, fast-prime, compilers and build tools remain third-party works under their respective licenses. Retain upstream notices when redistributing third-party material.

8. Clean buyer quick start

```
./setup_all.sh
./validate_all.sh
GOLD1_AC_QUALIFY_REPEATS=5 RANK_RUNS=5 ./run_interval_matrix.sh
```

That is the default handover path. No separate Visual Studio/Rust step is required unless the buyer specifically wants the optional fast-prime comparator.